

Writing a Strategy from Scratch — the Complete Guide

Quant Research Platform · strategy engine reference · September 2026

This guide explains, in full, how to write a trading strategy for the platform's Backtester and Optimizer. It assumes you have **never seen the application** and that nobody will be around to answer questions, so it states every rule explicitly, including the ones that seem obvious. Every code block is taken from files that are covered by the automated test suite.

Contents

1. The 60-second picture
 2. Quick start: your first strategy in 10 minutes
 3. How a run works (engine vs. strategy)
 4. The data
 5. Where a strategy lives: files, folders, imports
 6. The strategy contract — every name a strategy can declare
 7. Declaring data with `DATA`
 8. Parameters: `PARAMS`, `PARAM_SECTIONS`, `PARAMS_OPTIONS`
 9. `prepare_day` and `process_day`
 10. The `Day` object — reading today and previous days
 11. Producing trades: the `Trade` class and the output
 12. Time, fills and prices — the rules that prevent lies
 13. Caching explained
 14. Running a strategy: Backtester, saving trades, Optimizer
 15. Running and testing without the UI
 16. Worked example 1 — a single-file strategy
 17. Worked example 2 — a package with extra data and lookback
 18. Performance
 19. Every error and warning, with cause and fix
 20. Checklists
 21. FAQ
 22. Appendix A — instrument table (`ASSET_INFO`)
 23. Appendix B — glossary
-

1. The 60-second picture

A **strategy** is a Python file (or a folder of Python files) that answers one question: "Given the market data of one trading day, which trades would I have made?" That's all. The platform's **engine** does everything else:

The engine does	The strategy does
finds the day files in the chosen date range	declares which columns it needs (DATA)
reads only the declared columns from disk, a few days ahead, in the background	optionally pre-computes day data that never depends on parameters (prepare_day)
keeps the data in RAM between runs (the cache)	decides the trades of one day (process_day)
calls the strategy once per trading day, in date order	returns Trade objects
gives access to previous days (day.previous)	
builds the final trades table (12 fixed columns), computes pnl_points	
converts points to ticks, tags day types, shows the report	

The smallest possible strategy:

```
from modules.engine import Trade

PARAMS = {"tick_size": 0.25}
DATA = {"main": ["open", "close"]}

def process_day(day, params):
    bars = day.data["main"]                # this day's 1-minute bars
    first, last = bars.index[0], bars.index[-1]
    return Trade("long", first, last,      # buy the first open,
                float(bars["open"].iloc[0]), # sell the last close
                float(bars["close"].iloc[-1]), "eod")
```

Save it as **strategies/buy_the_session.py**, open the Backtester, pick a dataset and **buy_the_session**, and press **Run**.

2. Quick start: your first strategy in 10 minutes

1. **Create the file.** Make a new file in the repository's **strategies/** folder, e.g. **strategies/my_first.py**. The file name (without **.py**) becomes the name shown in the app, so use lowercase letters, digits and underscores. Never name it **__init__.py** or **base.py** (those names are skipped).
2. **Declare the parameters** in **PARAMS** (a dict of name → default value). Always include **"tick_size": 0.25** if you need the instrument's tick size; the app fills in the real value for the chosen asset.
3. **Declare the data** in **DATA: {"main": [list of column names]}**. See [section 4](#) for which columns each dataset has.
4. **Write process_day(day, params):** read **day.data["main"]** (a pandas DataFrame of today's bars), decide, and return **Trade(...)** objects (or **None** for "no trade today").
5. **Run it.** Start the app with **python main.py**, open **Backtester**, choose *Type* → *Asset* → *Dataset*, choose your strategy, set the dates, press **Run**. If the strategy is not in the list, press **Refresh folders**.
6. **Save the trades** with **Save Trades** below the report ([section 14](#)).

The rest of this guide explains each of those steps in complete detail.

3. How a run works (engine vs. strategy)

When you press **Run** in the Backtester (or when the Optimizer runs one parameter combination), the engine function **modules.engine.run_strategy** does exactly this, in this order:

1. **Validate the strategy.** It must have a **process_day** function and a **DATA** dict with a **"main"** key. **PREPARE_PARAMS**, if present, must be a list of parameter names, and then **prepare_day** must exist.
2. **Validate the data folders.** The main dataset folder must exist. Every additional **DATA** slot (every key other than **"main"**) must have a folder chosen for it, and that folder must exist.

3. **Build the parameters.** `params = {**PARAMS, **<values from the UI>, "tick_size": <from ASSET_INFO>}`. The UI's values override your defaults, and `tick_size` is always overwritten with the real value of the selected asset.
4. **List the days.** Every file named `YYYY-MM-DD.parquet` in the main dataset folder whose date is between the start and end date (both inclusive) is a day of the run. Other files are ignored. Days are processed in date order, oldest first.
5. **For each day:**
 1. If an additional slot has **no file for that date**, the day is **skipped** (and reported in a warning after the run).
 2. The engine gets `prepare_day`'s result for the day from the cache, or builds it by calling `prepare_day(date, data, prepare_params)` and caches it. If `prepare_day` returns `None`, the day is skipped silently.
 3. It calls `process_day(day, params)` and collects the returned trades, stamping each with the day's date.
 4. At **every step** (waiting for a disk read, preparing, running `process_day`, skipping) it reports what it is doing. The Backtester shows this live (see 14.1), and the **Cancel** buttons work through it: a cancelled run stops at the next step.
6. **Build the output.** The engine turns all `Trade` objects into one pandas DataFrame with exactly 12 columns (section 11).
7. **Report.** It returns a one-paragraph summary and a timing table (shown under the Backtester's progress bar, and printed to the console), plus the data warnings (skipped days), which the window shows in a yellow banner.

Days are independent. Within one run the days are processed in date order, but a parallel Optimizer run splits the date range into chunks that run in **different processes** (14.2). So `process_day` must never depend on anything an earlier `process_day` call left behind (module variables, attributes you set on the module). Everything a day needs from the past comes from `day.previous` / `day.previous_days(n)` (section 10), which work across chunk boundaries.

After the engine returns, the **Backtester** adds `ticks = pnl_points × ticks_per_point` and `cumulative_ticks`, tags each trade with a `day_type` (holiday / FOMC / CPI / ... from the economic calendar), and shows the report.

The key consequence: a strategy **never** opens files, globs folders, filters dates, loops over days, caches anything or builds DataFrames of trades. If you find yourself doing any of that, you are fighting the engine.

4. The data

4.1 Where the data lives

All market data lives under a **data root**, by default `D:/market_data` (the data roots are configured in the app's **Settings**, the gear icon on the main menu). The layout is fixed:

```
<data root>/
  parquet/
    {type}/           e.g. Futures
      {ASSET}/        an UPPERCASE ticker: ES, NQ, MES, CL, ...
        {dataset}/    e.g. ES_1m_advanced
          2026-09-21.parquet
          2026-09-22.parquet
          ...
  raw_dbn/...         raw Databento downloads (input of the Data Formatter)
  trades/             saved backtest trades
  optimizations/      saved optimizer runs
  regimes/             regime detector output
  news_and_holidays/  the economic calendar used for day types
```

A **dataset** is one folder containing **one parquet file per trading day**, named `YYYY-MM-DD.parquet`. The name is the **RTH date** (see 4.3): the file `2026-09-21.parquet` holds the Globex session that *starts* on the evening of Sunday 2026-09-20 and *ends* on the afternoon of Monday 2026-09-21. Files only exist for days with a regular trading session: Monday–Friday, minus exchange holidays. There are never files for Saturdays or Sundays.

4.2 The index: time zone, units, bar labels

Every day file is a pandas DataFrame whose **index is a time-zone-aware `DatetimeIndex` in `America/New_York`** (New York wall-clock time, with daylight-saving time handled). Rules you must know:

- **A bar is labelled by its OPEN time.** The 1-minute bar labelled `10:00` contains the trades from 10:00:00.000 to 10:00:59.999. Its `close` is the price of the last trade before 10:01:00. **You only know a bar's close once the bar has finished**, which is at the label of the *next* bar.
- **The index unit differs between datasets.** Some datasets store nanoseconds (`datetime64[ns, America/New_York]`), others microseconds (`datetime64[us, America/New_York]`). This is invisible when you compare the index with `pd.Timestamp` objects (always do this). It **matters** if you use `index.asi8` (raw integers): those are in the index's own unit, while `pd.Timestamp(...).value` is always nanoseconds. If you must use integers, convert first: `idx.as_unit("ns").asi8`.
- **Never compare the index with a naive timestamp.** Build comparison times with the index's own time zone: `pd.Timestamp(f"{day.date} 09:30", tz=bars.index.tz)`.
- The index has no name, except in `*_big_trades` (`ts_event`) and `*_1s_mbo_cropped` (`timestamp`).
- The index is sorted ascending, with no duplicates in bar datasets.

4.3 The session and the RTH date

- **Globex session:** the electronic session runs from **18:00 New York time on the previous calendar day** to **17:00** on the RTH date (with a daily maintenance break 17:00–18:00). For Monday, "the previous day" is Sunday.
- **RTH (regular trading hours): 09:30–16:00** New York time for equity index futures. Other markets have different pit hours; the data is still labelled by the same rule.
- **The file's date is the RTH date**, which is also `day.date` in your strategy.

The exact bar coverage differs between dataset kinds. This is the most common source of surprises:

Dataset kind	Bars per normal day	First bar	Last bar	Gaps
<code>{ASSET}_1m_ohlcv_globex</code>	always exactly 1380	18:00 (previous day)	16:59	minutes without trades are filled : open = high = low = close = previous close, volume = 0
<code>{ASSET}_1m_advanced</code>	1380 in summer (EDT), 1320 in winter (EST), fewer on half days	18:00 (previous day)	summer 16:59, winter 15:59 , half days at the last trade (e.g. 13:14 on 2025-11-28)	minutes between the first and last trade are filled the same way (volume 0, <code>tick_volume = "{}"</code>); nothing is added before the first or after the last trade
<code>{ASSET}_1m_indicators</code>	same index as the matching <code>_1m_advanced</code> day			
<code>{ASSET}_big_trades</code>	one row per large trade (not bars)	first large trade	last large trade	none — rows only where a trade happened
<code>{ASSET}_1s_mbo_cropped</code>	82,800 one-second bars (18:00 → 16:59:59)			filled

Practical rule: never assume a fixed number of bars or a bar at a fixed position. Always locate bars by time (`bars.index >= t`), and handle the case where the time you want doesn't exist (half days, winter `_1m_advanced` days, data gaps).

4.4 Dataset kinds and their columns

Values below are real, taken from the ES files of 2026-09-21. "Points" means the instrument's quote units (ES: index points; one point = 4 ticks of 0.25).

{ASSET}_1m_ohlcglobex — plain 1-minute OHLCV (all assets)

Column	dtype	Meaning	Example
open	float64	first trade price of the minute, points	7760.75
high	float64	highest trade price	7761.0
low	float64	lowest trade price	7760.5
close	float64	last trade price	7761.0
volume	int64	contracts traded in the minute	213

File metadata (not columns): **symbol/front_month** (the contract actually used, e.g. **ESZ6**), **trade_date**, **is_roll_day** ("True"/"False": the day the front month changed). Index unit: **microseconds**. Available from 2010 for ES.

{ASSET}_1m_advanced — 1-minute bars with order flow

Built from trade-by-trade data (Databento TBBO). The last two columns are only meaningful for liquid equity index futures (ES, NQ).

Column	dtype	Meaning	Example
open high low close	float64	as above	7760.75 ...
volume	uint32	contracts traded	213
buy_volume	int32	contracts bought by aggressive buyers (lifting the ask)	129
sell_volume	int32	contracts sold by aggressive sellers (hitting the bid)	84
volume_delta	int32	buy_volume - sell_volume	45
volume_delta_pct	float64	volume_delta / volume × 100, rounded to 0.1, always within -100...+100; 0.0 for zero-volume bars	21.1
tick_volume	str (JSON)	volume at each price in the bar, split by aggressor: {"price": [buy_qty, sell_qty], ...}	{"7760.5": [16, 21], "7760.75": [87, 60], "7761.0": [26, 3]}
passive_orders	str (JSON)	resting liquidity seen at trade time: {"price": [size, order_count], ...}. Buy aggressors record ask levels above the bar open; sell aggressors record bid levels below the bar open	{"7761.0": [25, 22], "7760.5": [8, 7]}

How to read the JSON columns (they are plain strings, parse them yourself; the keys are prices written as strings, like "7761.0"):

```
import json
levels = json.loads(bars["tick_volume"].iloc[i]) # {"7760.5": [16, 21], ...}
for price_str, (buy_qty, sell_qty) in levels.items():
    price = float(price_str)
```

An empty bar holds "{}". Parsing JSON is slow, so do it once per day in **prepare_day** (the result is cached), never inside a per-bar loop of **process_day**. File metadata: **front_month**, **trade_date**, **is_roll_day**. Index unit: **nanoseconds**. Available for ES/NQ from 2025-04-21 and for several other assets (see 4.6).

{ASSET}_1m_indicators — per-minute indicators

Same index as the matching **_1m_advanced** day. All float64. A value is **NaN** where it isn't defined yet (e.g. every *_**rth** column before 09:30).

Columns	Meaning
vwap_bar_globex , vwap_bar_globex_std{1,2,3}_{up,dn}	volume-weighted average price since 18:00 computed from the bars (typical price × volume), and its ±1/2/3 standard-deviation bands
vwap_bar_rth , vwap_bar_rth_std{1,2,3}_{up,dn}	the same, anchored at 09:30 (NaN before 09:30)

Columns	Meaning
<code>vwap_tick_globex</code> , <code>vwap_tick_globex_std{1,2,3}_{up,dn}</code>	VWAP computed from the exact per-price volume in <code>tick_volume</code> (more precise)
<code>vwap_tick_rth</code> , <code>vwap_tick_rth_std{1,2,3}_{up,dn}</code>	the same, anchored at 09:30
<code>cumulative_delta</code>	running sum of <code>volume_delta</code> since 18:00 (CVD)

29 columns in total. The `vwap_tick_*` and `cumulative_delta` columns exist only where the source bars had order flow; the file simply omits them otherwise. VWAP values are raw floats, **not** rounded to the tick grid.

`{ASSET}_big_trades` — individual large trades

One row per trade of at least N contracts (N = the transform's thresholds when the dataset was built; 10 by default, separately settable before and during RTH), front-month contract only, over the Globex session. Days without any large trade have no file.

Column	dtype	Meaning	Example
<code>index ts_event</code>	<code>datetime64[ns, America/New_York]</code>	exact trade time (nanoseconds)	2026-05-25 12:59:50.000619349
<code>price</code>	<code>float64</code>	trade price, points	7558.0
<code>size</code>	<code>uint32</code>	contracts	11
<code>side</code>	<code>str</code>	aggressor: "B" = buyer lifted the ask, "A" = seller hit the bid	"A"

`{ASSET}_1s_mbo_cropped` — one-second bars with order-book depth

Available for ES and NQ, built from the full order book (MBO). Columns: `open high low close` (`float64`), `volume` (`int64`), `buy_volume` `sell_volume` (`int32`), `best_bid` `best_ask` (`float64`), and three JSON string columns: `aggressor_volume` (the second's aggressive volume by price; `"{}"` when nothing traded), `bid_depth` and `ask_depth` (`{"price": resting_size, ...}`, the book cropped around the market; the file metadata records the crop settings `n_ticks`, `big_order_mult` and `baseline_window_min`). Very large (82,800 rows per day), so declare only what you need and parse in `prepare_day`.

`{ASSET}_statistics` — NOT a day dataset

This folder holds a **single** file, `statistics.parquet` (one row per session: settlement, session and RTH OHLC, volumes). It is **not** one file per day, so it **cannot** be a **DATA** slot. It is used by the report's Market Exposure section.

4.5 Where the data comes from

Raw Databento downloads (`.dbn.zst`) go into `raw_dbn/{type}/{ASSET}/{folder}/`. The app's **Data Formatter** module turns them into parquet datasets with a *data transform* (plugins in `data_transforms/`):

Transform	Input (Databento schema)	Produces
<code>1m_ohlc_v_globex_single_asset / 1m_ohlc_v_globex_mixed_assets</code>	<code>ohlcv-1m</code>	<code>{ASSET}_1m_ohlcv_globex</code>
<code>1m_advanced</code>	TBBO (trades with the top of book)	<code>{ASSET}_1m_advanced</code>
<code>1m_advanced_indicators</code>	<code>a_1m_advanced</code> (or OHLCV) parquet dataset	<code>{ASSET}_1m_indicators</code>
<code>big_trades</code>	TBBO	<code>{ASSET}_big_trades</code>
<code>1s_mbo_cropped</code>	MBO (full order book)	<code>{ASSET}_1s_mbo_cropped</code>
<code>settlement_reference</code>	<code>statistics</code>	<code>{ASSET}_statistics</code>

To make a new dataset: put the raw files in `raw_dbn/...`, open the Data Formatter, choose the input folder, the transform and an output folder name, and run it. Output lands in `parquet/{type}/{ASSET}/{name}/`.

4.6 Which assets have which datasets (September 2026)

Asset	Datasets
ES	1m_advanced, 1m_indicators, 1m_ohlcglobex, 1s_mbo_cropped, big_trades, statistics
NQ	1m_advanced, 1m_indicators, 1m_ohlcglobex, 1s_mbo_cropped, big_trades, statistics
ZN	1m_advanced, 1m_indicators, 1m_ohlcglobex
6B 6C 6E CL GC MES NG NNQ SR3	1m_advanced, 1m_ohlcglobex
6J BTC HG HO M2K MGC MNQ MYM QM RB RTY SI YM ZB ZC ZF ZS ZT ZW	1m_ohlcglobex

The dataset folder name always starts with the ticker: `ES_1m_advanced`, `NQ_1m_indicators`, and so on.

4.7 Front-month contracts and rolls

Each day file contains the **front-month** contract only: the contract with the highest volume that session. When the front month changes (a *roll*), prices jump between days by the spread between the contracts. So compare prices within a day freely, but be careful comparing prices **across** days around roll dates (the files' `is_roll_day` metadata marks them). Within one day there is never a contract switch.

5. Where a strategy lives: files, folders, imports

5.1 Discovery

The app scans the **strategy folders** every time the Backtester/Optimizer opens or you press **Refresh folders**. The folders are: the repository's `strategies/` folder, plus any extra folders added in **Settings** → **Strategy folders**. In each folder:

- every file `name.py` is a strategy called `name`, except `__init__.py` and `base.py`;
- every **sub-folder** that contains an `__init__.py` is a **package strategy** called by the folder name;
- if the same name exists in two folders, both appear, labelled `name [folder]`.

A strategy must define a `process_day` function; otherwise the app reports "Strategy 'name' has no `process_day(day, params)` function (see `STRATEGY_GUIDE.md`)" when you select it.

5.2 Single file vs package

- Single file** (`strategies/my_strategy.py`): best for up to ~200 lines.
- Package** (`strategies/my_strategy/` with `__init__.py` + other modules): for anything larger. `__init__.py` must expose the contract names (`PARAMS`, `DATA`, `process_day`, ...); the logic can live in other modules of the folder, which `__init__.py` imports **relatively**:

```
# strategies/my_strategy/__init__.py
from .params import PARAMS, PARAM_SECTIONS    # relative import: note the dot
from .logic import process_day
DATA = {"main": ["open", "high", "low", "close"]}
```

5.3 Import rules

Do	Don't
<code>from modules.engine import Trade, timed</code>	copy the <code>Trade</code> class into your strategy
relative imports inside a package: <code>from .logic import x</code>	<code>from base import x</code> , <code>from strategies.x import y</code> , or <code>sys.path</code> tricks — they break when the strategy lives in an extra folder or runs in an Optimizer worker
<code>import numpy as np, import pandas as pd, import json, import math</code>	import anything from the app's UI (PySide6, <code>modules.*.window</code> , <code>modules.common.ui</code>) — Optimizer workers must stay GUI-free
define plain classes and functions	put <code>from __future__ import annotations</code> in a strategy file that defines a <code>@dataclass</code> (it crashes the plugin loader on Python 3.13)

Available libraries: Python 3.13, pandas 3.0, numpy 2.4, pyarrow 23. pandas 3 uses the `str` dtype for text columns by default.

5.4 The strategy module is re-loaded

The app executes your strategy file **again** every time you select it, and each Optimizer worker process loads it once. Module-level variables therefore **reset** at those moments. Never rely on module-level state surviving between runs; the engine's cache is the only thing that does. (A module-level *memo of a pure function*, such as validated parameters, is fine.)

6. The strategy contract — every name a strategy can declare

Name	Required	Type	What it is
<code>process_day</code>	yes	<code>function (day, params) -> Trade list[Trade] None</code>	decides one day's trades (section 9)
<code>DATA</code>	yes	<code>dict[str, list[str] "*"]</code>	the datasets and columns to load; must contain "main" (section 7)
<code>PARAMS</code>	practically yes	<code>dict[str, default]</code>	parameter names and defaults; drives the form (section 8)
<code>PARAM_SECTIONS</code>	no	<code>dict[str, list[str]]</code>	groups parameters into collapsible boxes
<code>PARAMS_OPTIONS</code>	no	<code>dict[str, list]</code>	dropdown / checkbox-group choices
<code>prepare_day</code>	no	<code>function (date, data, params) -> object None</code>	per-day pre-computation, cached (section 9)
<code>PREPARE_PARAMS</code>	no (only with <code>prepare_day</code>)	<code>list[str]</code>	the parameters <code>prepare_day</code> depends on

Nothing else is read. Other module-level names are yours to use freely (names in `__all__` are purely documentation).

7. Declaring data with `DATA`

7.1 The shape

```
DATA = {
    "main": ["open", "high", "low", "close", "volume"], # the dataset picked at the top
    "indicators": ["vwap_bar_rth", "cumulative_delta"], # an ADDITIONAL dataset
}
```

- Keys are **slot names**. "main" is mandatory and always means the dataset chosen in the Backtester's *Type / Asset / Dataset* pickers.
- Every other key is an **additional data slot**: a second (third, ...) dataset read for the same date. Slot names are any non-empty strings. Use lowercase words that appear in the dataset folder names (`indicators`, `big_trades`, `options`), because they are used to auto-pick the folder (7.3).
- Each value is a **non-empty list of column names** (no duplicates), or the string "*" meaning "every column in the file".

7.2 What the engine does with it

- It reads **only** the listed columns from each day file, in the listed order. Unlisted columns are never read. That's why declaring only what you need makes runs faster and saves RAM.
- `day.data["main"]` returns a pandas DataFrame with exactly those columns and the file's own index.
- **Declared column missing from a file** → the run stops with: *"Column(s) ['tick_volume'] declared in DATA['main'] are not in the 'main' dataset ES_1m_ohlc_v_globex (file 2026-03-02.parquet). Pick a dataset that has them, or remove them from DATA."*
- **Using a column you did not declare** (e.g. `day.data["main"]["volume"]` when "volume" isn't listed) → pandas raises `KeyError`, which the engine turns into a message that starts *"KeyError 'volume' in the day 2026-03-02 of 'my_strategy'. If 'volume' is a data column, it is not loaded — the strategy declares DATA = {...}; add it to the right slot..."*

- **Unknown slot** (`day.data["options"]` when `"options"` isn't a key of `DATA`) → "No data slot 'options' — the strategy declares DATA slots ['main', 'indicators']."

7.3 Additional slots in the UI

For every additional slot, the Backtester and the Optimizer show one row in an **"Additional data"** section, with its own *Type* → *Asset* → *Dataset* pickers:

- *Type* and *Asset* start equal to the main selection and follow it when you change the main pickers.
- *Dataset* is **auto-picked**: the first dataset of that type/asset whose folder name **contains the slot name**. The comparison ignores case, underscores and dashes, so slot `indicators` finds `ES_1m_indicators`, and slot `big_trades` (or `BigTrades`) finds `ES_big_trades`. A dataset in the same data root as the main dataset is preferred.
- You can change any row by hand, including picking **another asset**. For example, an MES backtest can use ES indicators.
- A row with **no matching dataset** shows "[!] no matching dataset", and **Run is refused** with: "No dataset for 'indicators' under Futures/MES — no folder name contains 'indicators'. Pick one by hand or create it with the Data Formatter."

Every additional slot is **required**. There is no "optional data".

7.4 Days with a missing additional file

The main dataset defines the days. If an additional slot's folder has **no file for one of those dates**, that day is **skipped entirely**: no `prepare_day`, no `process_day`, no trades. After the run a **yellow warning banner** lists them:

```
[!] The 'indicators' dataset (ES_1m_indicators) has no file for 12 day(s), so those days were SKIPPED: 2026-03-05, 2026-03-06, ... (+2 more)
```

The same text is printed to the console and, for Optimizer runs, saved in the run's `meta.json` under `data_warnings`. A result computed on fewer days must never look complete, so treat this banner seriously: rebuild the missing files with the Data Formatter.

Days where the **main** dataset has no file are not days of the run at all (the main dataset defines the calendar).

7.5 Using an additional slot in code

```
def process_day(day, params):
    bars = day.data["main"]
    ind = day.data["indicators"]          # same date, declared columns only
    # the index of ind is the file's own index – align by time, not by position:
    vwap = ind["vwap_bar_rth"].reindex(bars.index)
```

Two datasets of the same day do **not** necessarily have identical indexes (for example `_1m_ohlcv_globex` has 1380 bars, while `_1m_indicators` follows the `_1m_advanced` grid, which has 1320 bars in winter). **Always align by timestamp** (`reindex`, `join`), never by row position.

8. Parameters: PARAMS, PARAM_SECTIONS, PARAMS_OPTIONS

8.1 PARAMS

A dict of **parameter name** → **default value**. The **type of the default** decides the input widget in the Backtester, what `params` contains, and how the Optimizer can sweep it:

Default type	Widget	Value you receive	Optimizer sweep
<code>bool</code> (True/False)	checkbox	<code>bool</code>	over [<code>False</code> , <code>True</code>]
<code>int</code> (e.g. 15)	integer spin box (–1,000,000,000 ... 1,000,000,000)	<code>int</code>	min / max / step range
<code>float</code> (e.g. 0.5)	decimal spin box (±1e12, 2 decimals shown)	<code>float</code>	min / max / step range

Default type	Widget	Value you receive	Optimizer sweep
str (e.g. "09:30")	text box	str	comma-separated list of values
a value listed in PARAMS_OPTIONS[name]	dropdown	the chosen option, with its original type	a subset of the options
a "0"/"1" string as long as PARAMS_OPTIONS[name]	one named checkbox per option	the bit string (e.g. "1011")	a comma-separated list of bit strings
anything else (list, dict, None)	a warning label, not editable	the default, unchanged	not sweepable

Rules:

- **Use real True/False for switches**, not 0/1: an int gets a number box and an int sweep.
- **Floats need a float default**. Write 2.0, not 2, if the value can be fractional; an int default gives an int-only box.
- **Times are strings** ("09:30"), and your code parses them. The engine gives no special time widget.
- **Parameter names must be unique** and must not collide with trade columns (date, direction, trade_type, entry_time, exit_time, entry_price, exit_price, sl, tp, exit_reason, pnl_points, notes, pnl_ticks, day_bucket). The Optimizer refuses to sweep a parameter with such a name.
- The params dict your functions receive always contains **every** key of PARAMS (defaults overridden by the form) **plus tick_size**.

8.2 tick_size (automatic)

tick_size is an **automatic parameter**. Declare it in PARAMS with any float default (use 0.25). Then:

- the form shows it **greyed out** (read-only), with the real value of the selected asset from ASSET_INFO (Appendix A), updated whenever you change the asset;
- the Optimizer shows it read-only and **never sweeps** it;
- the engine **always** puts the real value into params["tick_size"], even if you did not declare it.

Use it to convert tick-based parameters to points (stop_ticks * params["tick_size"]) and to round prices to the tick grid (section 12.4).

8.3 PARAM_SECTIONS

```
PARAM_SECTIONS = {
    "Range": ["tick_size", "range_minutes"],
    "Risk": ["stop_buffer_ticks", "target_rr", "exit_time"],
}
```

Each key is a collapsible box in the form, containing the listed parameters in that order. A parameter not listed in any section appears in a final box called **"Other"**, so a typo in a name silently moves it there. Without PARAM_SECTIONS, all parameters appear in plain rows. With several sections, the boxes start collapsed; with one section, it starts open.

8.4 PARAMS_OPTIONS

```
PARAMS = {"direction_filter": "both", "entries": "110"}
PARAMS_OPTIONS = {
    "direction_filter": ["both", "long_only", "short_only"], # default is in the list -> dropdown
    "entries": ["breakout", "retest", "reversal"], # "110" has 3 chars -> 3 checkboxes
}
```

- **Dropdown**: when the default value is one of the listed choices. Choices may be str, int or float, and you receive the chosen value with its original type.
- **Checkbox group (bit string)**: when the default is a string of "0"/"1" with exactly one character per option. The form shows one named checkbox per option and returns the bit string ("101" = first and third checked). Read bit *i* with params["entries"][i] == "1".
- A bool default always renders as a checkbox, whatever PARAMS_OPTIONS says.
- PARAMS_OPTIONS constrains the **UI only**: nothing stops code (or an old saved run) from passing another value, so validate in your code if it matters.

8.5 Validating parameters

Validate early and raise `ValueError` with a clear message. The run then stops and the message appears in the red error banner. Because `process_day` runs once per day, memoize validation per parameter set:

```
_CFG = {}

def _config(params):
    key = tuple(sorted(params.items()))      # params values are hashable (str/int/float/bool)
    cfg = _CFG.get(key)
    if cfg is None:
        hh, mm = map(int, params["exit_time"].split(":"))
        if not (0 <= hh <= 23 and 0 <= mm <= 59):
            raise ValueError(f"exit_time must be HH:MM, got {params['exit_time']!r}")
        cfg = {"exit_minute": hh * 60 + mm}
        _CFG[key] = cfg
    return cfg
```

9. prepare_day and process_day

9.1 process_day(day, params) — required

- **Called:** once per day of the run, in date order, on one thread.
- **day:** a `Day` object (section 10).
- **params:** a plain `dict` with every parameter (section 8).
- **Returns** one of:
 - `None`: no trade today;
 - a `Trade`;
 - a list (or any iterable) of `Trade` objects, in the order they should appear (any number per day).
- Returning anything else (a dict, a DataFrame, a tuple of numbers) stops the run with *"process_day of 'my_strategy' returned a dict — every trade must be a modules.engine.Trade."*
- Exceptions raised inside `process_day` stop the run and show in the red banner (with the traceback in the console).

9.2 prepare_day(date, data, params) — optional, cached

Use it for work that depends **only on the day's data** (and on the few parameters listed in `PREPARE_PARAMS`): slicing the session, converting columns to numpy arrays, parsing the JSON columns, computing daily levels.

- **date:** `datetime.date`, the day's RTH date.
- **data:** a plain `dict` slot → DataFrame, with **all** slots of that day (`data["main"]`, `data["indicators"]`, ...), declared columns only.
- **params:** a dict containing **only** the parameters named in `PREPARE_PARAMS` (an empty dict when there are none).
- **Returns** any Python object: a class instance, a dict, a tuple, numpy arrays (the "prepared day"). Returning `None` **skips the day** (no `process_day` for it): use this for unusable days (too few bars, empty file). Earlier-day lookups then see `prepared == None` for that day.
- The result is **cached**: the next runs (any parameters), every Optimizer combination, and later days' `day.previous` lookups reuse it without calling `prepare_day` again. See section 13.
- `process_day` reads it as `day.prepared`.

9.3 PREPARE_PARAMS

A list of parameter names that change what `prepare_day` builds, e.g. `PREPARE_PARAMS = ["rth_start"]` if the session start is a parameter. These parameters (and only these) are passed to `prepare_day` and are **part of the cache key**, so a different `rth_start` builds and caches a separate prepared day. **If `prepare_day` depends on a parameter that is not in `PREPARE_PARAMS`, the cache returns stale results computed with a different value.** The engine can't detect this. The safe rule: `prepare_day` may only read `params[...]` for names in `PREPARE_PARAMS`, and the engine enforces this by passing only those.

9.4 What goes where

Work	Where	Why
slicing the RTH session, <code>to_numpy()</code> conversions	<code>prepare_day</code>	same for every parameter set, so cached once
parsing <code>tick_volume</code> / <code>passive_orders</code> JSON	<code>prepare_day</code>	expensive, so do it once per day, ever
daily summaries (RTH high/low/close, range, volume profile)	<code>prepare_day</code>	reused by the next days' lookback for free
anything using a sweepable parameter (thresholds, windows, RR...)	<code>process_day</code>	must be recomputed per parameter set
building the <code>Trade</code> objects	<code>process_day</code>	

9.5 Never modify cached objects

The DataFrames in `day.data`, and whatever `prepare_day` returned, are **shared**: the same objects are served to the next run, to other strategies reading the same file, and to the next days' lookback. **Never modify them in place.** For example, never do any of these:

```
bars["mid"] = (bars["high"] + bars["low"]) / 2    # WRONG: adds a column to the cached frame
core.close[5] = 0.0                             # WRONG: writes into a cached array
core.levels.append(x)                           # WRONG: grows a cached list every run
```

Copy first when you need modified data: `bars = day.data["main"].copy()`, or `mid = (bars["high"] + bars["low"]) / 2` as a separate Series.

The one accepted exception is an object that **resets** its own per-run state first thing in every `process_day` call. The `ivb_model` strategy does this with `day.reset_run_state()`, because rebuilding its big per-day structure would be too slow. If you use this pattern, the reset must restore *everything* a previous run could have written.

10. The Day object — reading today and previous days

10.1 Reference

Member	Type	Meaning
<code>day.date</code>	<code>datetime.date</code>	the RTH date of this day (the file name)
<code>day.data[slot]</code>	<code>pandas.DataFrame</code>	the declared columns of slot <code>slot</code> for this date; loaded on first access (from the cache or the disk)
<code>day.data.keys()</code>	<code>list[str]</code>	the strategy's slot names
<code>day.prepared</code>	object / <code>None</code>	what <code>prepare_day</code> returned for this day; <code>None</code> if the strategy has no <code>prepare_day</code>
<code>day.previous</code>	<code>Day</code> / <code>None</code>	the previous trading day : the previous file in the main dataset folder. <code>None</code> for the very first file.
<code>day.previous_days(n)</code>	<code>list[Day]</code>	up to <code>n</code> earlier trading days, oldest first (so the last element is <code>day.previous</code>). Fewer than <code>n</code> near the start of the data; <code>[]</code> for <code>n <= 0</code> .
<code>day.missing</code>	<code>list[str]</code>	additional slots with no file for that day. Always <code>[]</code> for the day being processed (incomplete days are skipped); check it before reading a slot of an earlier day.

10.2 How lookback works

- A previous day is a **full Day**: it has **data** for every slot and its own **prepared**, and you can chain `day.previous.previous`.
- Lookback uses **every file of the main dataset folder**, not just the run's date range, so the first day of a run still sees the days before the start date.
- **Trading days, not calendar days**: Monday's **previous** is the previous Friday (or the last trading day before a holiday).
- It is **lazy** and **cached**: nothing is read until you touch it, and what you touch is kept in RAM. Because the engine processes days in order, yesterday is almost always already in RAM.
- A previous day whose additional file is missing raises *"The 'indicators' data file for 2026-03-05 does not exist: ..."* when you read that slot (or its **prepared**). Check `d.missing` first.
- A previous day for which `prepare_day` returned `None` has `prepared == None`.

10.3 Examples

```
# yesterday's RTH high/low from the raw bars
prev = day.previous
if prev is not None:
    pb = prev.data["main"]
    t0 = pd.Timestamp(f"{prev.date} 09:30", tz=pb.index.tz)
    t1 = pd.Timestamp(f"{prev.date} 16:00", tz=pb.index.tz)
    rth = pb[(pb.index >= t0) & (pb.index < t1)]
    prev_high, prev_low = rth["high"].max(), rth["low"].min()

# average daily volume of the last 5 trading days
days = day.previous_days(5)
avg_volume = sum(d.data["main"]["volume"].sum() for d in days) / len(days) if days else None

# yesterday's final CVD (additional slot), skipping a day without the file
prev = day.previous
if prev is not None and not prev.missing:
    cvd_close = prev.data["indicators"]["cumulative_delta"].iloc[-1]

# the best way: summarise each day ONCE in prepare_day, then read summaries
history = [d.prepared for d in day.previous_days(20) if not d.missing]
history = [h for h in history if h is not None]
```

The last pattern is the fastest: the summaries are computed once per day and cached, so a 20-day lookback costs almost nothing.

11. Producing trades: the Trade class and the output

11.1 Trade

```
from modules.engine import Trade

Trade(
    direction="long",                # required: "long" or "short" (lowercase)
    entry_time=bars.index[e],        # required: tz-aware pd.Timestamp of the entry fill
    exit_time=bars.index[k],         # required: tz-aware pd.Timestamp of the exit fill
    entry_price=5559.5,              # required: float, points
    exit_price=5531.75,              # required: float, points
    exit_reason="sl",                # required: short label
    sl=5531.75,                      # optional: stop price, points (default NaN)
    tp=5614.5,                      # optional: target price, points (default NaN)
    trade_type="prev_day_high_break", # optional: label (default None)
    notes={"prev_high": 5554.5},     # optional: dict (default None)
)
```

Field	Type	Required	Default	Meaning / rules
<code>direction</code>	<code>str</code>	yes	—	exactly "long" or "short" ; anything else raises <i>"Trade.direction must be 'long' or 'short', got 'Long'"</i> immediately
<code>entry_time</code>	<code>pd.Timestamp</code>	yes	—	tz-aware (America/New_York); normally a bar label from your data

Field	Type	Required	Default	Meaning / rules
exit_time	pd.Timestamp	yes	—	tz-aware; must not be before <code>entry_time</code>
entry_price	float	yes	—	the fill price, points
exit_price	float	yes	—	the exit fill price, points
exit_reason	str	yes	—	why it closed; the report groups by it (conventions below)
sl	float	no	NaN	the stop level (points) as placed; used by the R:R statistics
tp	float	no	NaN	the target level (points) as placed
trade_type	str None	no	None	a label, e.g. which setup fired; the report can filter by it
notes	dict None	no	None	extra facts about the trade (11.4)

Positional arguments work in the order of the table (`Trade("long", t0, t1, 100.0, 101.0, "eod")`), but keywords are clearer.

11.2 What the engine adds

- **date**: the RTH date of the day whose `process_day` returned the trade (a `datetime.date`), even if the trade's times fall in the previous evening.
- **pnl_points**: computed, never supplied: `exit_price - entry_price` for a long, `entry_price - exit_price` for a short. No commissions or slippage (those are applied later by Analytics / Monte Carlo per contract).

11.3 The output table

Every run returns a pandas DataFrame with **exactly these 12 columns, in this order**, even when there are no trades (then it has 0 rows):

#	Column	dtype	Source
1	date	object (<code>datetime.date</code>)	engine
2	direction	str	<code>Trade.direction</code>
3	trade_type	str / object (<code>None</code>)	<code>Trade.trade_type</code>
4	entry_time	datetime64[unit, America/New_York]	<code>Trade.entry_time</code> (unit follows your timestamps)
5	exit_time	datetime64[unit, America/New_York]	<code>Trade.exit_time</code>
6	entry_price	float64	<code>Trade.entry_price</code>
7	exit_price	float64	<code>Trade.exit_price</code>
8	sl	float64	<code>Trade.sl</code>
9	tp	float64	<code>Trade.tp</code>
10	exit_reason	str	<code>Trade.exit_reason</code>
11	pnl_points	float64	engine
12	notes	str (JSON) / object (<code>None</code>)	<code>json.dumps(Trade.notes)</code>

Rows are in day order, and within a day in the order you returned them. The Backtester then adds `ticks`, `cumulative_ticks` and `day_type`.

11.4 notes

`notes` is a **flat dict of simple values** describing the trade. The engine stores it as `json.dumps(notes)`, keeping your key order. Each strategy chooses its own keys; different trades may have different keys. The report's **Trades & Notes** table shows every key as a column (`notes.<key>`), and you can filter the whole report by note values.

Allowed value types: `str`, `int`, `float`, `bool`, `None`, and lists/dicts of those. **Not allowed**: numpy scalars (`np.float64`, `np.int64`, `np.bool_`), Timestamps, dates, numpy arrays. Convert with `float(x)`, `int(x)`, `bool(x)`, `ts.strftime("%H:%M")`, `str(date)`, `arr.tolist()`. A non-serializable value raises `TypeError: Object of type int64 is not JSON serializable` when the output is built. `NaN` is technically allowed but is written as the non-standard JSON token `NaN`, so prefer `None` for "no value".

A real example (ivb_model):

```
{"breakout_time": "10:22", "retest_time": "10:39", "flip_count": 0, "ivb_high": 5524.75,
"ivb_low": 5490.75, "poc": 5500.5, "vah": 5522.75, "val": 5500.5,
"absorption_time": ["10:53", "10:54"], "two_bar_baseline": 105.21, "trigger_price": 5512.5,
"trigger_volume": 242, "tp_type": "tp_vwap_2", "escalated": false, "trail_count": 0}
```

11.5 exit_reason conventions

Use short lowercase labels. The existing strategies use: **tp** (target), **sl** (stop), **eod** (end of session), **time_exit** / **timeout_profit** / **tp_timeout** / **sl_timeout** (time-based), **trailing_sl** (trailed stop), **vwap_flip**, **band_flat**, **exclusion**. Any string works; the report groups identical labels.

12. Time, fills and prices — the rules that prevent lies

A backtest is only as honest as its fills. These rules are conventions of the platform; the engine can't enforce them, so it is your job.

12.1 No look-ahead

At the time of bar i , you may only use information from bars $0 \dots i-1$ plus bar i 's open. Concretely:

- A signal computed from bar i 's **close** (or high, low, volume) is known only when bar i has finished. The **earliest possible entry is the open of bar $i+1$** . All example strategies do this: **$e = s + 1$** .
- Never enter at the close of the bar whose close created the signal.
- Never use a day-level value (the day's high, the final VWAP, the last row of an indicator) to decide a trade earlier that same day. Previous days' values are fine.
- An indicator row is valid at the **end** of its bar, the same as the close.

12.2 Intrabar order

A 1-minute bar doesn't tell you whether its high or its low came first. When a single bar touches both the stop and the target, assume the **stop** was hit first (pessimistic), as all example strategies do: check the stop before the target inside each bar.

12.3 The entry bar itself

After entering at bar e 's open, bar e 's own high/low may already hit the stop or target. The examples check exits starting **at the entry bar** (**`for k in range(e, ...)`**). That's correct because the open comes first.

12.4 Points, ticks and the tick grid

- All prices you give the engine are in **points** (the instrument's own price units). Never convert to ticks yourself; the Backtester does it with **`ticks = pnl_points * ticks_per_point`**.
- Real orders can only rest at prices on the **tick grid** (ES: multiples of 0.25). Computed stops and targets (e.g. "entry - 0.25 × average range") usually fall between ticks. Round them, **away from the entry for stops** (never a tighter stop than you meant) and **toward the entry for targets** (never a target beyond what you meant):

```
import math
tick = params["tick_size"]
if direction == "long":
    sl = math.floor((entry - stop_dist) / tick) * tick
    tp = math.floor((entry + stop_dist * rr) / tick) * tick
else:
    sl = math.ceil((entry + stop_dist) / tick) * tick
    tp = math.ceil((entry - stop_dist * rr) / tick) * tick
```

- Prices from the data (opens, closes, highs, lows) are already on the grid.
- For float noise: **`0.1 + 0.2 != 0.3`**. Compare prices with a tolerance of half a tick when equality matters.

12.5 One position at a time?

The engine doesn't manage positions. Each **Trade** is an independent round trip; you may return overlapping trades if your logic means it, and each counts as one contract. Use Analytics' position sizing to scale.

13. Caching explained

13.1 Why

Reading parquet from disk is the slowest part of a backtest. The engine keeps day data **in RAM** ("floating") between runs, so the second run of a strategy (any parameters), every Optimizer combination after the first, and other strategies reading the same files skip the disk. Measured on ES_1m_advanced + ES_1m_indicators, 372 days, ivb_model: **cold run $\approx$ 8 s, warm run $\approx$ 1.8 s** (including the report), with $\approx$ **0.29 GB** held in RAM.

13.2 What is cached

One cache per process, with one budget, holding two kinds of entries:

Entry	Key (what makes it unique)	Value
raw	file path + file modification time + file size + the declared column list	the DataFrame read from the file
prepared	strategy file path + date + (path, modification time, size) of every slot's file + declared columns + the values of <code>PREPARE_PARAMS</code>	what <code>prepare_day</code> returned

Consequences:

- Changing a **normal parameter** reuses everything (prepared and raw).
- Changing a **PREPARE_PARAMS parameter** rebuilds the prepared days, but reuses the raw frames (no disk reads).
- **Rewriting a data file** (e.g. re-running a transform) changes its modification time, so it is re-read automatically. Nothing stale.
- Changing **DATA** (columns) creates new entries automatically.
- **Changing the CODE of `prepare_day` is NOT detected.** The cache would keep serving days built by the old code. **After editing `prepare_day` (or anything it calls), press "Free cached data", or restart the app.**
- Changing `process_day` needs nothing: it runs every time.

13.3 The budget and eviction

- **Backtester** (and Optimizer runs with 1 worker): the budget is **Settings $\rightarrow$ Engine day cache $\rightarrow$ RAM budget (GB)**, default **4 GB**. It applies at the start of each run.
- **Optimizer with N parallel workers:** each worker is a separate process with its own cache. The **Memory budget (GB)** field of the Optimizer's New Run tab is the **total** for all workers: each gets $\text{budget} \div N$. That fits, because each worker only ever holds its own $1/N$ chunk of the days (14.2): the data is split between the workers, never copied. Worker caches disappear when the run ends.
- When a cache is full, entries are dropped in this order:
 1. **raw frames whose day is already prepared and cached** (they're only needed to rebuild it, and they're about two thirds of the cached bytes);
 2. entries the **current run hasn't used**, least recently used first;
 3. if everything left is in use by the current run, the **new** entry is simply not kept.

Rule 3 makes the cache *scan-resistant*: a run walks the days in order, and a plain "drop the oldest" cache smaller than the run would throw away exactly the days the next run (the next Optimizer combination) needs first, so every run would be cold. Keeping the first days instead means a too-small cache still serves most of them.

- When a run couldn't keep everything, the Backtester and the Optimizer show a yellow note: *"The RAM cache (0.07 GB) was too small for this run: 276 item(s) could not be kept and will be read or prepared again next run. Raise the budget (Settings $\rightarrow$ Engine day cache; the Optimizer's Memory budget for parallel workers) or shorten the date range."* The same text is printed to the console, saved in an Optimizer run's `meta.json` as `cache_note`, and returned as `RunResult.cache_note` / `cache_rejected`.
- Sizes are **estimates** (DataFrames exactly as pandas' deep memory usage; prepared objects by walking their numpy arrays, DataFrames, strings and containers). A prepared object can report its own size precisely by defining a method `__cache_nbytes__(self) -> int`. A prepared day built during a run is measured again **once, right after its first `process_day`**, so objects that fill lazy caches on first use (ivb_model parses its JSON

columns that way) are counted at their real size.

13.4 "Free cached data"

The **Free cached data** button (Backtester, next to Run; Optimizer New Run tab, next to Run grid) empties the cache of the app process and shows how much it freed. Use it:

- after changing `prepare_day` or code it calls (mandatory, see 13.2);
- to give RAM back to other programs;
- when comparing cold-run timings.

13.5 Sizing tips

Per trading day, measured in RAM: a 1-minute Globex day with 4 OHLC columns is ~0.06 MB; `ES_1m_advanced` with `ivb_model`'s columns (incl. the `tick_volume` / `passive_orders` JSON) ~0.5 MB and `NQ_1m_advanced` ~2.2 MB; a prepared `ivb_model` day with its parsed JSON ~1.9 MB (NQ). The 1-second MBO datasets are in another league: 4 OHLC columns of `ES_1s_mbo_cropped` are ~3 MB per day, but **all** columns (`DATA = {"main": "*"}`, with the `bid_depth` / `ask_depth` book JSON) are **~225 MB per day** — 136 days need ~30 GB. Declare only the columns you need (the JSON columns are by far the largest), and let `prepare_day` keep compact numpy arrays rather than DataFrames and strings. The Optimizer's New Run tab shows an estimate for the selected strategy, dataset and dates.

14. Running a strategy: Backtester, saving trades, Optimizer

14.1 Backtester, step by step

1. **Main menu** → **Backtester**. Each click opens an independent window.
2. **Type / Asset / Dataset**: the main data (`DATA["main"]`). Start/End date are limited to the dataset's first and last file.
3. **Strategy**: your strategy. Its parameter form appears below, with `tick_size` greyed out.
4. **Additional data** (only if your `DATA` has extra slots): check the auto-picked rows.
5. **Run**. A progress bar (`day i / n`) appears with what the engine is doing **right now**, updated live (see "The progress panel" below). **Cancel** (next to Run while a run is going) stops after the current step. Errors appear in a red banner, and data warnings in a yellow one.
6. **The report**: metrics, equity curve with drawdown, breakdowns by exit reason, by day type (holiday/FOMC/CPI/NFP/...), by regime, R:R distribution, market exposure, and the Trades & Notes table. Filters at the top apply to the whole report.
7. **Save Trades** writes the currently filtered trades to `<data root>/trades/{dataset}_{strategy}_{start}_{end}.parquet`, e.g. `ES_1m_advanced_ivb_model_2025-04-21_2026-09-25.parquet`. A filtered save gets `_filtered` added, and a name collision gets `_2`, `_3`, Saving identical trades twice is refused. The file contains your 12 columns plus `ticks` and `cumulative_ticks` (`day_type` and regime columns are removed), with the active filters recorded in the parquet metadata. The first part of the name must stay the ticker: Analytics and Monte Carlo read the asset from it.
8. **Go to Analytics / Go to Monte Carlo** open those modules on the current trades (via a temporary file in `<data root>/temp/`).

The progress panel. During a run, four lines under the progress bar say what the engine is doing, for example:

```
Day 123 of 372 · 2025-10-09 – preparing the day (prepare_day)
Reading ahead on 2 background threads: 2025-10-10 ... 2025-10-15
So far: 57 trades · 122 days processed · files read from disk: main 126, indicators 126 · files from RAM: 0 ·
    prepared days: 123 built, 0 from RAM
RAM cache 0.11 / 4.00 GB · elapsed 2.1 s · about 4.3 s left
```

The step at the end of the first line is one of: `waiting for the disk: reading main + indicators`, `preparing the day (prepare_day)`, `prepared day found in RAM`, `running the strategy (process_day)`, or `skipped: no indicators file for this day`. Before and after the day loop the first line

reads **Checking ...**, **Listing the day files of ...**, **Found N days in range ...** and **Building the trades table from N trades...** When the run ends, the panel shows the engine's summary (days, trades, files read from disk vs from RAM, prepared days built vs reused, cache fill) and a collapsible **Engine timing of the last run** table (14.3).

14.2 Optimizer basics

The Optimizer runs your strategy once per **combination** of up to **4 swept parameters** (roles: X axis, Y axis, slider, slider 2) and shows a heatmap of any metric.

- Tick **sweep** on a parameter to vary it (the widgets follow 8.1); every other parameter stays at the value shown. **tick_size** can't be swept.
- More than **2,000** combinations requires an extra confirmation.
- **Parallel workers**: 1 = everything in the app process, sharing the Backtester's cache; the log shows one line per combination (with where its data came from) and, during a slow combination, one line a second saying which day the engine is on. $N > 1 = N$ processes, and the **date range is split into N contiguous chunks of days**, one per worker. Every worker runs **every** combination, but only on its own chunk. So each day file is read and prepared exactly once, by one worker (plus up to a few **previous_days** at a chunk's first day), and even a single combination runs N times faster. The workers' results are joined in date order into exactly the table a 1-worker run produces. Each worker's cache is capped at **Memory budget** $\div N$, which holds its $1/N$ of the days.
- **Parallel workers start empty on every run**. They are new processes, so they can't see data the app already holds in RAM (from the Backtester or earlier runs). The first combination therefore loads each worker's chunk from disk (the log says **worker 3/8 (2025-09-02 ... 2025-10-21, 46 days) loaded its data in 1.9s: 92 files read from disk, 46 days prepared**), and every later combination runs from RAM. Because the chunks load in parallel, that first load takes about $1/N$ of a cold Backtester run.
- Additional data rows work exactly as in the Backtester.
- A saved run (`<data root>/optimizations/{run}/`) contains **trades.parquet** (all combinations' trades, with the swept parameters as extra columns) and **meta.json** with: **strategy**, **dataset**, **ticker**, **tick_size**, **ticks_per_point**, **dates**, **axes**, **fixed_params**, **additional_data** (`{slot: "type/asset/dataset"}`), **data_warnings**, counts and creation time.

14.3 The console output

Every Backtester run prints (in the terminal that started the app) the same summary and timing table the Backtester shows under **Engine timing of the last run**:

```
[engine] ivb_model: 372 days in range, 372 processed, 176 trades in 4.5 s. Files read from disk: 744 (main 372,
    indicators 372; 5.10 s of reading on 2 threads), from RAM: 0. Prepared days: 372 built, 0 from RAM. RAM cache
    0.37 / 4.00 GB.
[ivb_model timing] wall 4.521s
  section                total s   calls   ms/call   % wall
  day:process            2.790    372     7.500    61.7%
  bg io:read:main        3.210    372     8.629    71.0%
  day:prepare            1.180    372     3.172    26.1%
  ...
```

The engine's own sections:

Section	Thread	What it measures
engine:setup	main	checking DATA , the folders and the params
engine:list_days	main	listing the main dataset's day files
engine:read_plan	main	deciding the read-ahead: reads the first day's parquet footers; days whose declared columns are bigger than 32 MB in RAM (full-book MBO) get 4 read threads and 6 days of read-ahead, everything else 2 and 4
engine:schedule	main	deciding which files a day needs read (includes engine:file_stat)
engine:file_stat	main	looking up a file's size and modification time (once per file per run)
bg io:read:<slot>	background	reading one day file of a slot (only the declared columns)
bg cache:store	background	measuring and storing a frame that was just read

Section	Thread	What it measures
<code>io:wait_for_read</code>	main	the day loop waiting for a background read to finish (the disk is the bottleneck when this is large)
<code>io:read:<slot></code>	main	a read on the main thread (a <code>day.previous</code> whose files were not read ahead)
<code>cache:lookup / cache:store</code>	main	cache access for prepared days
<code>cache:remeasure</code>	main	re-measuring a prepared day after its first <code>process_day</code> (13.3)
<code>day:prepare</code>	main	your <code>prepare_day</code>
<code>day:process</code>	main	your <code>process_day</code> (your own <code>timed</code> sections nest inside it)
<code>engine:collect_trades</code>	main	checking and collecting the returned <code>Trades</code>
<code>engine:progress</code>	main	building the progress text (only when something shows it)
<code>build_output_df</code>	main	building the 12-column trades table

Background (`bg`) sections run at the same time as the main thread, so the `% wall` column adds up to more than 100%.

Use `timed` to add your own sections to the table (sections nest, so the percentages overlap):

```
from modules.engine import timed

def process_day(day, params):
    with timed("my:signal_scan"):
        ...
```

15. Running and testing without the UI

15.1 A script

Run from the repository root (so that `modules` is importable). The `if __name__ == "__main__":` guard is **mandatory** on Windows as soon as the Optimizer uses parallel workers (each worker re-imports the launching script), and it's good practice anyway.

```
# run_my_strategy.py (in the repository root)
from pathlib import Path

from modules.engine import run_strategy
from modules.optimizer.backend.loader import load_strategy

def main():
    root = Path("D:/market_data/parquet/Futures/ES")
    strategy = load_strategy("example_prev_day_levels") # by name, from strategies/
    result = run_strategy(
        strategy,
        root / "ES_1m_advanced", # the main dataset folder
        "2025-06-01", "2026-06-01", # start, end (inclusive)
        {"target_rr": 1.5}, # overrides of PARAMS
        tick_size=0.25, # normally from ASSET_INFO
        extra_folders={"indicators": root / "ES_1m_indicators"},
    )
    print(result.trades.tail())
    print(result.warnings, result.days_in_range, result.days_run,
          result.files_read, result.days_prepared)

if __name__ == "__main__":
    main()
```

`run_strategy` returns a `RunResult` with: `trades` (the 12-column DataFrame), `warnings` (list of str), `days_in_range`, `days_run`, `skipped` (`{slot: [dates]}`), `files_read` (disk reads), `days_prepared` (`prepare_day` calls), `cache_rejected / cache_note` (how many items the RAM cache was too full to keep), `elapsed` (seconds), `summary` (the one-paragraph text), `timing` (the timing table as text) and `rows` (the (`date`, `Trade`) pairs behind `trades`). Useful keyword arguments: `cache=DayCache()` (a private cache, e.g. for tests), `verbose=False` (no console output), `on_progress=callback(day_i, n_days, text)` (called at every step; `text` is the four lines shown in 14.1, and raising an exception inside the callback cancels the run).

15.2 A test with synthetic data

Build a few tiny days in a temporary folder, run the strategy, and assert the exact trade you computed by hand. This is how both worked examples are tested (`tests/test_strategy_examples.py`):

```
import pandas as pd
from modules.engine import DayCache, run_strategy
from modules.optimizer.backend.loader import load_strategy

TZ = "America/New_York"

def make_day(folder, date, base=100.0, bars=None):
    idx = pd.date_range(f"{date} 09:30", f"{date} 15:59", freq="1min", tz=TZ)
    df = pd.DataFrame({"open": base, "high": base + 0.1, "low": base - 0.1,
                      "close": base}, index=idx, dtype=float)
    for hhmm, (o, h, l, c) in (bars or {}).items():
        df.loc[pd.Timestamp(f"{date} {hhmm}", tz=TZ)] = [o, h, l, c]
    folder.mkdir(parents=True, exist_ok=True)
    df.to_parquet(folder / f"{date}.parquet")

def test_my_strategy(tmp_path):
    make_day(tmp_path / "ES_1m", "2026-03-02", bars={"10:40": (100, 101.6, 100, 101.5)})
    result = run_strategy(load_strategy("example_first_hour_breakout"),
                          tmp_path / "ES_1m", "2026-01-01", "2026-12-31", {},
                          tick_size=0.25, cache=DayCache(), verbose=False)
    assert len(result.trades) == 1
```

Run the tests with `python -m pytest tests -q`.

16. Worked example 1 — a single-file strategy

`strategies/example_first_hour_breakout.py`: trades the first close outside the first hour's range. It shows the minimal structure: `PARAMS`, `PARAM_SECTIONS`, `DATA`, `process_day`, time-based slicing, next-bar entry, stop-first exits, tick conversion, and notes.

```
"""
example_first_hour_breakout – the minimal single-file strategy from
STRATEGY_GUIDE.md (worked example 1). Copy it as a starting template.

Rules (one trade per day at most):
1. The "first hour" is the RTH bars from 09:30 up to (not including)
   09:30 + range_minutes (NY time). Its high/low form the range.
2. The first later bar that CLOSES above the range high is a long signal,
   below the range low a short signal. We enter at the NEXT bar's open
   (never at the signal bar's own close – that price is already history
   when the bar is known to have closed).
3. Stop = the opposite side of the range, pushed stop_buffer_ticks further
   away. Target = entry +/- (entry-to-stop distance) x target_rr.
4. If neither is hit, exit at the close of the last bar at or before
   exit_time. Within one bar the stop is checked first (pessimistic).
"""

import numpy as np
import pandas as pd

from modules.engine import Trade

PARAMS = {
    "tick_size": 0.25, # auto-filled from ASSET_INFO (read-only in the UI)
    "range_minutes": 60, # length of the opening range in minutes
    "stop_buffer_ticks": 2, # extra ticks beyond the range for the stop
    "target_rr": 1.5, # target distance = stop distance x this
    "exit_time": "15:55", # flat at the close of the last bar at/before this (NY)
}

PARAM_SECTIONS = {
    "Range": ["tick_size", "range_minutes"],
    "Risk": ["stop_buffer_ticks", "target_rr", "exit_time"],
}

DATA = {"main": ["open", "high", "low", "close"]}

def process_day(day, params):
    candles = day.data["main"] # this day's bars, declared columns only
    tz = candles.index.tz # America/New_York
```

```

d = day.date                                # datetime.date of the RTH session

rth_open = pd.Timestamp(f"{d} 09:30", tz=tz)
range_end = rth_open + pd.Timedelta(minutes=params["range_minutes"])
exit_at = pd.Timestamp(f"{d} {params['exit_time']}", tz=tz)

# Compare the index with Timestamps (works for ns- and us-unit indexes).
idx = candles.index
in_range = (idx >= rth_open) & (idx < range_end)
after = (idx >= range_end) & (idx <= exit_at)
if not in_range.any() or after.sum() < 2:
    return None                                # half day / missing data: no trade

rng = candles[in_range]
hi, lo = float(rng["high"].max()), float(rng["low"].min())

bars = candles[after]
o = bars["open"].to_numpy()
h = bars["high"].to_numpy()
l = bars["low"].to_numpy()
c = bars["close"].to_numpy()

# signal on a CLOSE beyond the range; the last bar can't signal (no next open)
long_sig = c[:-1] > hi
short_sig = c[:-1] < lo
either = long_sig | short_sig
if not either.any():
    return None
s = int(np.argmax(either))                    # first signal bar
direction = "long" if long_sig[s] else "short"

e = s + 1                                    # entry bar = the bar after the signal
entry = float(o[e])
buffer = params["stop_buffer_ticks"] * params["tick_size"]
if direction == "long":
    sl = lo - buffer
    tp = entry + (entry - sl) * params["target_rr"]
else:
    sl = hi + buffer
    tp = entry - (sl - entry) * params["target_rr"]
if (direction == "long" and sl >= entry) or (direction == "short" and sl <= entry):
    return None                                # entry already beyond the stop

# walk the bars from the entry bar on; stop first within a bar
for k in range(e, len(c)):
    if direction == "long":
        if l[k] <= sl:
            return _trade(direction, bars, e, k, entry, sl, "sl", sl, tp, hi, lo)
        if h[k] >= tp:
            return _trade(direction, bars, e, k, entry, tp, "tp", sl, tp, hi, lo)
    else:
        if h[k] >= sl:
            return _trade(direction, bars, e, k, entry, sl, "sl", sl, tp, hi, lo)
        if l[k] <= tp:
            return _trade(direction, bars, e, k, entry, tp, "tp", sl, tp, hi, lo)
last = len(c) - 1
return _trade(direction, bars, e, last, entry, float(c[last]), "time_exit", sl, tp, hi, lo)

def _trade(direction, bars, e, k, entry, exit_price, reason, sl, tp, hi, lo):
    return Trade(
        direction=direction,
        entry_time=bars.index[e],
        exit_time=bars.index[k],
        entry_price=entry,
        exit_price=float(exit_price),
        exit_reason=reason,
        sl=float(sl),
        tp=float(tp),
        trade_type="first_hour_breakout",
        notes={"range_high": hi, "range_low": lo, "bars_held": int(k - e + 1)},
    )

```

Walk-through:

- **DATA = {"main": [...]}:** four columns, so the engine reads nothing else, and it works on any dataset kind (`_1m_ohlcv_globex` or `_1m_advanced`).

- **Time boundaries are `pd.Timestamps` in the index's time zone**, built from `day.date`. That makes the comparisons correct for both ns and us indexes and across daylight-saving changes.
- `after.sum() < 2` guards half days and missing data: at least one signal bar and one entry bar must exist.
- `c[:-1]`: the last bar can't be a signal, because there'd be no next bar to enter on.
- `e = s + 1`: entry at the open of the bar after the signal (section 12.1).
- `buffer = stop_buffer_ticks * tick_size`: a tick-based parameter converted to points with the automatic `tick_size`.
- **Stop checked before target in every bar**, starting at the entry bar (12.2, 12.3).
- **Notes** contain only Python floats/ints (`float(...)`, `int(...)`).

Verified behaviour (from the tests): with a first-hour range 99–101 and a 10:40 close at 101.5, the trade is long at 10:41's open 101.5, stop 98.5 (99 minus 2 ticks), target 106.0 (risk 3.0×1.5), pnl +4.5 points when the target is hit.

17. Worked example 2 — a package with extra data and lookback

`strategies/example_prev_day_levels/`: trades a break of **yesterday's RTH high or low**, filtered by RTH VWAP and the direction of CVD (from the indicators dataset), with a stop sized by the **average RTH range of the last 5 days**. It shows an additional **DATA** slot, `prepare_day` with `PREPARE_PARAMS`, `previous_days`, `missing`, tick-grid rounding, a dropdown parameter and richer notes.

`params.py`

```
"""Parameters of example_prev_day_levels."""

PARAMS = {
    "tick_size": 0.25, # auto-filled from ASSET_INFO (read-only in the UI)
    "rth_start": "09:30", # RTH session start "HH:MM" NY (a PREPARE_PARAM)
    "signal_after": "10:00", # no signals before this bar (NY)
    "exit_time": "15:55", # flat at the close of the last bar at/before this (NY)
    "lookback_days": 5, # days averaged for the typical RTH range
    "stop_range_frac": 0.25, # stop distance = average RTH range x this
    "target_rr": 2.0, # target distance = stop distance x this
    "require_vwap": True, # longs only above RTH VWAP, shorts only below
    "direction_filter": "both", # "both", "long_only" or "short_only"
}

PARAM_SECTIONS = {
    "Session": ["tick_size", "rth_start", "signal_after", "exit_time"],
    "Levels": ["lookback_days", "require_vwap", "direction_filter"],
    "Risk": ["stop_range_frac", "target_rr"],
}

PARAMS_OPTIONS = {
    "direction_filter": ["both", "long_only", "short_only"],
}
```

day.py — the prepared day

```
"""prepare_day's result: one RTH session as numpy arrays + its summary.

Everything here depends only on the day's files and `rth_start` (declared in
PREPARE_PARAMS), so the engine caches it and reuses it for every later run,
every optimizer combo, and for `day.previous` lookups from the next days.
Nothing in here may be modified after it is built.
"""

import numpy as np
import pandas as pd

RTH_END = "16:00"

class RthDay:
    """The RTH bars [rth_start, 16:00) of one day."""
    __slots__ = ("index", "open", "high", "low", "close", "vwap", "cvd",
                 "rth_high", "rth_low", "rth_close", "rth_range")

    def __init__(self, candles: pd.DataFrame, indicators: pd.DataFrame, date, rth_start: str):
        tz = candles.index.tz
        start = pd.Timestamp(f"{date} {rth_start}", tz=tz)
        end = pd.Timestamp(f"{date} {RTH_END}", tz=tz)
        rth = candles[(candles.index >= start) & (candles.index < end)]
        # indicators are aligned to the candles by timestamp (same 1-minute grid)
        ind = indicators.reindex(rth.index)

        self.index = rth.index
        self.open = rth["open"].to_numpy(dtype=np.float64)
        self.high = rth["high"].to_numpy(dtype=np.float64)
        self.low = rth["low"].to_numpy(dtype=np.float64)
        self.close = rth["close"].to_numpy(dtype=np.float64)
        self.vwap = ind["vwap_bar_rth"].to_numpy(dtype=np.float64)
        self.cvd = ind["cumulative_delta"].to_numpy(dtype=np.float64)
        self.rth_high = float(self.high.max())
        self.rth_low = float(self.low.min())
        self.rth_close = float(self.close[-1])
        self.rth_range = self.rth_high - self.rth_low

    def build(date, candles: pd.DataFrame, indicators: pd.DataFrame, rth_start: str):
        """RthDay, or None when the day has fewer than 30 RTH bars (the engine then
        skips the day, and later days see it as a previous day with prepared=None)."""
        tz = candles.index.tz
        start = pd.Timestamp(f"{date} {rth_start}", tz=tz)
        end = pd.Timestamp(f"{date} {RTH_END}", tz=tz)
        n = int(((candles.index >= start) & (candles.index < end)).sum())
        if n < 30:
            return None
        return RthDay(candles, indicators, date, rth_start)
```

logic.py — signals and exits

```
"""The per-day trading logic: previous-day high/low breakout, VWAP- and
CVD-filtered, with a stop sized by the average RTH range of the last days."""

import numpy as np
import pandas as pd

from modules.engine import Trade

def find_signal(core, prev, params):
    """(bar_index, direction) of the first bar at/after `signal_after` that
    closes beyond yesterday's RTH high (long) / low (short) and passes the
    filters – or None. The last bar can't signal (the entry is the NEXT open)."""
    tz = core.index.tz
    date = core.index[0].date()
    t0 = pd.Timestamp(f"{date} {params['signal_after']}", tz=tz)
    t_exit = pd.Timestamp(f"{date} {params['exit_time']}", tz=tz)
    allow_long = params["direction_filter"] in ("both", "long_only")
    allow_short = params["direction_filter"] in ("both", "short_only")

    for i in range(1, len(core.close) - 1):
        t = core.index[i]
        if t < t0:
            continue
        if t >= t_exit:
            return None
        c, v = core.close[i], core.vwap[i]
        cvd_up = core.cvd[i] > core.cvd[i - 1]
        if allow_long and c > prev.rth_high and cvd_up \
            and (not params["require_vwap"] or (not np.isnan(v) and c > v)):
            return i, "long"
        if allow_short and c < prev.rth_low and not cvd_up \
            and (not params["require_vwap"] or (not np.isnan(v) and c < v)):
            return i, "short"
    return None

def simulate(core, e, direction, sl, tp, params, notes):
    """Walk from the entry bar `e`: stop first within a bar, then target, then
    the time exit at the close of the last bar at/before `exit_time`."""
    tz = core.index.tz
    date = core.index[0].date()
    t_exit = pd.Timestamp(f"{date} {params['exit_time']}", tz=tz)
    entry = float(core.open[e])
    last = e
    for k in range(e, len(core.close)):
        if core.index[k] > t_exit:
            break
        last = k
        if direction == "long":
            if core.low[k] <= sl:
                return _trade(core, e, k, direction, entry, sl, "sl", sl, tp, notes)
            if core.high[k] >= tp:
                return _trade(core, e, k, direction, entry, tp, "tp", sl, tp, notes)
        else:
            if core.high[k] >= sl:
                return _trade(core, e, k, direction, entry, sl, "sl", sl, tp, notes)
            if core.low[k] <= tp:
                return _trade(core, e, k, direction, entry, tp, "tp", sl, tp, notes)
    return _trade(core, e, last, direction, entry, float(core.close[last]),
        "time_exit", sl, tp, notes)

def _trade(core, e, k, direction, entry, exit_price, reason, sl, tp, notes):
    return Trade(
        direction=direction,
        entry_time=core.index[e],
        exit_time=core.index[k],
        entry_price=entry,
        exit_price=float(exit_price),
        exit_reason=reason,
        sl=float(sl),
        tp=float(tp),
        trade_type=f"prev_day_{'high' if direction == 'long' else 'low'}_break",
        notes={**notes, "bars_held": int(k - e + 1)},
    )
```

`__init__.py` — the contract

```
"""
example_prev_day_levels – the package strategy from STRATEGY_GUIDE.md
(worked example 2). Shows an additional data slot, prepare_day with
PREPARE_PARAMS, looking back over previous days, and rich notes.

Rules (one trade per day at most):
1. Yesterday's RTH high/low are the levels (from day.previous).
2. From `signal_after` on, the first bar that CLOSES above yesterday's high
   with CVD rising (and above RTH VWAP when require_vwap) is a long signal;
   below yesterday's low with CVD falling (and below VWAP) a short signal.
3. Entry = the NEXT bar's open. Stop distance = the average RTH range of the
   last `lookback_days` days x stop_range_frac; target = stop distance x
   target_rr. Time exit at `exit_time`.
4. Days without enough history (fewer than lookback_days usable previous
   days) are not traded.

Files:
  params.py  PARAMS / PARAM_SECTIONS / PARAMS_OPTIONS
  day.py     prepare_day's RthDay (cached by the engine)
  logic.py   signal detection + exit simulation -> Trade
"""

import math

from .day import build
from .logic import find_signal, simulate
from .params import PARAM_SECTIONS, PARAMS, PARAMS_OPTIONS

DATA = {
    "main": ["open", "high", "low", "close"],
    "indicators": ["vwap_bar_rth", "cumulative_delta"],
}

PREPARE_PARAMS = ["rth_start"]

def prepare_day(date, data, params):
    return build(date, data["main"], data["indicators"], params["rth_start"])

def process_day(day, params):
    core = day.prepared
    # earlier days, oldest first; skip days missing an additional-data file
    # (reading them would raise) and days prepare_day rejected (None)
    history = [d.prepared for d in day.previous_days(params["lookback_days"])
               if not d.missing]
    history = [h for h in history if h is not None]
    if len(history) < params["lookback_days"]:
        return None # not enough history yet
    prev = history[-1] # the most recent usable day
    avg_range = sum(h.rth_range for h in history) / len(history)

    sig = find_signal(core, prev, params)
    if sig is None:
        return None
    s, direction = sig
    e = s + 1 # entry at the next bar's open
    entry = float(core.open[e])
    stop_dist = avg_range * params["stop_range_frac"]
    if stop_dist <= 0:
        return None
    # prices must sit on the instrument's tick grid: round the stop AWAY from
    # the entry (never tighter than intended) and the target TOWARD it
    tick = params["tick_size"]
    if direction == "long":
        sl = math.floor((entry - stop_dist) / tick) * tick
        tp = math.floor((entry + stop_dist * params["target_rr"]) / tick) * tick
    else:
        sl = math.ceil((entry + stop_dist) / tick) * tick
        tp = math.ceil((entry - stop_dist * params["target_rr"]) / tick) * tick

    notes = {
        "prev_date": str(prev.index[0].date()), # the day the levels come from
        "prev_high": prev.rth_high,
        "prev_low": prev.rth_low,
        "avg_range": round(avg_range, 2),
        "signal_time": core.index[s].strftime("%H:%M"),
    }
```

```

        "vwap_at_entry": None if math.isnan(core.vwap[e]) else float(core.vwap[e]),
    }
    return simulate(core, e, direction, sl, tp, params, notes)

__all__ = ["DATA", "PREPARE_PARAMS", "PARAMS", "PARAM_SECTIONS", "PARAMS_OPTIONS",
           "prepare_day", "process_day"]

```

Walk-through:

- **Two slots.** **DATA** asks for 4 candle columns and 2 indicator columns. In the Backtester, pick an `_1m_advanced` dataset as the main data; the "indicators" row auto-picks `{ASSET}_1m_indicators`. On an asset without an indicators dataset (e.g. MES), pick ES indicators by hand or the run is refused.
- `PREPARE_PARAMS = [{"rth_start"}]`: the prepared day depends on the session start, so it is part of the cache key, and `prepare_day` receives only `{"rth_start": ...}`.
- **RthDay is built once per day and cached.** It holds compact arrays and the day's RTH summary (`rth_high`, `rth_low`, `rth_range`). The next days read those summaries through `day.previous_days(5)` without touching the disk.
- **build returns None for days with fewer than 30 RTH bars**, so those days are skipped and appear as `prepared == None` in later lookbacks. That's why `process_day` filters `h` is not `None`.
- `if not d.missing`: a previous day whose indicators file is missing is excluded instead of raising.
- **Indicators are aligned by timestamp** (`indicators.reindex(rth.index)`), never by row position (section 7.5).
- **The signal uses closes** of bars `1 ... n-2`, and the entry is the next bar's open. The CVD comparison uses bar `i` and `i-1` only, so there's no look-ahead.
- **Stops and targets are rounded to the tick grid** as described in 12.4.
- `direction_filter` is a dropdown (`PARAMS_OPTIONS`); the Optimizer can sweep a subset of its three choices.

Verified behaviour (from the tests): with five history days of RTH range 100–110 and a 10:30 close at 111 on the sixth day (CVD rising, above VWAP 105), the trade is long at 111.0, stop 108.5 ($111 - 10 \times 0.25$), target 116.0, with notes `prev_date` = the previous day, `prev_high` 110.0, `avg_range` 10.0, `signal_time` "10:30".

18. Performance

Do:

- **Declare only the columns you use.** The JSON columns are ~95% of an `_1m_advanced` file's size.
- **Move param-independent work into `prepare_day`:** slicing, `to_numpy()`, JSON parsing, daily summaries. It runs once per day, ever, until the cache is freed.
- **Work on numpy arrays** (`bars["close"].to_numpy()`) and vectorized comparisons (`closes > level`, `np.argmax(mask)` for "first True") instead of `DataFrame.iterrows()`, which is 100× slower. A plain Python loop over numpy values is fine for exits (a few hundred iterations).
- **Find times with `index.searchsorted(ts)`** or boolean masks, not by looping.
- **Memoize parameter validation** (section 8.5).
- **Use `timed(...)`** to see where time goes.

Don't:

- Don't call `pd.read_parquet` yourself. Declare the data instead.
- Don't parse JSON per bar inside `process_day`.
- Don't build DataFrames per bar or per trade.
- Don't copy whole day frames unless you modify them.

19. Every error and warning, with cause and fix

Messages are shown in the window's red banner (errors) or yellow banner (warnings), and printed in the console. `name` is your strategy's name, `slot` a **DATA** key.

Message (verbatim)	Cause	Fix
Strategy 'name' has no process_day(day, params) function (see STRATEGY_GUIDE.md)	no process_day at module level (package: not exported by __init__.py)	define it, or import it in __init__.py
Strategy 'name' not found in <folder>	name typo in a script, or the file was moved	check the name / folder
Strategy 'name' must declare DATA = {"main": [columns...], ...} (got None).	no DATA, or no "main" key	add DATA with "main"
Strategy 'name': DATA['slot'] must be a non-empty list of column names or "*" (got ...).	empty list, a single string, non-strings	["col1", "col2"] or "*"
Strategy 'name': DATA['slot'] lists a column twice.	duplicate column	remove the duplicate
Strategy 'name': DATA keys must be non-empty strings (got ...).	a key like "" or 1	use a word
Strategy 'name': PREPARE_PARAMS must be a list of param names.	a string instead of a list	["rth_start"]
Strategy 'name' declares PREPARE_PARAMS but has no prepare_day().	PREPARE_PARAMS without prepare_day	add prepare_day or remove PREPARE_PARAMS
PREPARE_PARAMS ['x'] of 'name' are not params.	a PREPARE_PARAMS name missing from PARAMS	add it to PARAMS
'name' needs additional data 'slot', but no dataset was chosen for it.	a script called run_strategy without extra_folders[slot]	pass the folder
The dataset chosen for 'slot' does not exist: <path>	wrong path / deleted folder	pick another dataset
The main dataset folder does not exist: <path>	wrong path	fix the path
No dataset for 'slot' under Type/ASSET – no folder name contains 'slot'. Pick one by hand or create it with the Data Formatter.	the asset has no dataset whose name contains the slot name	pick one in the row, or build it
Column(s) [...] declared in DATA['slot'] are not in the 'slot' dataset <dataset> (file <date>.parquet). Pick a dataset that has them, or remove them from DATA.	the chosen dataset lacks a declared column (e.g. tick_volume in an OHLCV dataset)	choose a dataset kind that has it (section 4.4)
KeyError 'col' in the day <date> of 'name'. If 'col' is a data column, it is not loaded – ...	you read a column not in DATA, or a param not in PARAMS	declare it
No data slot 'x' – the strategy declares DATA slots [...]	day.data["x"] for an undeclared slot	add the slot to DATA or fix the name
The 'slot' data file for <date> does not exist: <path>	reading a slot (or prepared) of an earlier day that lacks the file	check d.missing first
process_day of 'name' returned a dict – every trade must be a modules.engine.Trade.	returned dicts / tuples	return Trade(...)
process_day of 'name' must return a Trade, a list of Trades or None (got int).	returned a non-iterable	return None, a Trade or a list
Trade.direction must be 'long' or 'short', got 'Long'	wrong case / value	lowercase "long"/"short"
Object of type int64 is not JSON serializable	a numpy value in notes	wrap with int() / float()
[!] The 'slot' dataset (<dataset>) has no file for N day(s), so those days were SKIPPED: ... (warning)	missing files in an additional dataset	rebuild them with the Data Formatter; until then, the result excludes those days
worker pool died while running 'name' – usually the strategy failed to load in a worker or a worker ran out of memory (...)	the strategy crashes on import in a worker, a worker ran out of RAM, or a script without the __main__ guard	test the strategy in the Backtester first; lower workers / memory; add the guard

20. Checklists

Before the first run

- [] The file is in `strategies/` (or a Settings strategy folder), named `lowercase_name.py` (or a package folder with `__init__.py`).
- [] `from modules.engine import Trade` (no copied class, no `from base import`).
- [] `PARAMS` has correctly typed defaults (`2.0` for floats, `True` for switches, `"09:30"` for times) and `"tick_size": 0.25` if you use ticks.
- [] `DATA["main"]` lists exactly the columns you read, and they exist in the dataset kind you'll pick (section 4.4).
- [] Every additional slot's name appears in the dataset folder name you want auto-picked.
- [] `process_day` returns `None`, a `Trade`, or a list of `Trade`.
- [] Times are built with `tz=bars.index.tz` from `day.date`.

Honesty

- [] Entries are at the **next** bar's open after the signal bar.
- [] No same-day future information (day high/low, final values) is used before it existed.
- [] Stop is checked before target in each bar.
- [] Stops/targets are on the tick grid (rounded in the safe direction).
- [] Half days, winter `_1m_advanced` days (end 15:59) and missing times are handled.

Caching

- [] `prepare_day` only uses `PREPARE_PARAMS` parameters.
- [] Nothing modifies `day.data` frames or `day.prepared` in place.
- [] After changing `prepare_day`: **Free cached data**.

Output

- [] `notes` values are plain Python types.
- [] `exit_reason` labels are consistent.
- [] Prices are points, directions lowercase.

21. FAQ

Can a strategy trade more than once per day? Yes: return a list of `Trades`.

Can a trade last several days? Not across `process_day` calls: each call only sees its own day (and earlier days). Model overnight holds by entering and exiting within one day's data. A Globex day already spans the evening session of the previous calendar day (18:00 → 17:00).

Can I keep state from one day to the next (e.g. an open position)? No. Don't use module variables: they reset whenever the strategy is reloaded, and a parallel Optimizer run gives each worker process its own chunk of the days, so the day before a chunk's first day was processed in another process. Derive what you need from earlier days with `day.previous` / `previous_days` instead; that's deterministic, cached, and works across chunks.

How do I use 5- or 15-minute bars? Either pick a dataset with that bar size as the main data, or resample 1-minute bars inside `prepare_day` (cached): `bars.resample("15min").agg({"open": "first", "high": "max", "low": "min", "close": "last"}).dropna()`. Remember that a resampled bar is labelled by its open time and is only complete at its end.

How do I use two timeframes at once? Resample in `prepare_day` and keep both, or declare a second slot whose dataset has the other timeframe.

Can an additional slot come from another asset? Yes: change the row's `Asset` in the UI (e.g. ES indicators for an MES backtest). Prices of different contracts are in their own units.

Where do commissions and slippage come in? Not in the strategy. `pnl_points` is the pure price difference; Analytics and Monte Carlo apply per-contract commissions from `ASSET_INFO`.

Why does my strategy not appear in the list? Check the file name (not `__init__`/`base`), that it's in a strategy folder, then press **Refresh folders**. A strategy that fails to import shows an error when you select it.

Why are results different after I edited `prepare_day`? You probably haven't pressed **Free cached data**: the old prepared days were still being served.

Does the order of `PARAMS` matter? Only for display (within a section, or when there are no sections).

What time zone is `day.date` in? It's a plain date (the RTH date), not a timestamp. Combine it with times using the index time zone.

Can I print from a strategy? Yes; output goes to the console that started the app. For timing, use `timed`.

Appendix A — instrument table (`ASSET_INFO`)

From `modules/common/backend/asset_info.py`. `tick_size` is what your strategy receives as `params["tick_size"]`. The Backtester computes ticks as `pnl_points × ticks_per_point`.

Ticker	tick_size	ticks_per_point	\$ per tick	commission / side	parent
ES	0.25	4	12.5	2.88	
NQ	0.25	4	5	2.88	
RTY	0.1	10	5	2.88	
YM	1	1	5	2.88	
MES	0.25	4	1.25	0.95	ES
MNQ	0.25	4	0.5	0.95	NQ
NNQ	0.5	2	0.1	—	NQ
M2K	0.1	10	0.5	0.95	RTY
MYM	1	1	0.5	0.95	YM
ZN	0.015625	64	15.625	2.3	
ZB	0.03125	32	31.25	2.37	
ZF	0.0078125	128	7.8125	2.15	
ZT	0.00390625	256	7.8125	2.15	
SR3	0.0025	400	6.25	2.1	
CL	0.01	100	10	3	
QM	0.025	40	12.5	2.7	
NG	0.001	1000	10	3.1	
RB	0.0001	10000	4.2	3	
HO	0.0001	10000	4.2	3	
GC	0.1	10	10	3.1	
MGC	0.1	10	1	1.2	GC
SI	0.005	200	25	3.1	
HG	0.0005	2000	12.5	3.1	
ZC	0.25	4	12.5	3.6	
ZS	0.25	4	12.5	3.6	
ZW	0.25	4	12.5	3.6	
6E	0.00005	20000	6.25	3.1	
6J	0.0000005	2000000	6.25	3.1	
6B	0.0001	10000	6.25	3.1	
6C	0.00005	20000	5	3.1	
BTC	5	0.2	25	8	

parent links a micro/nano contract to the full-size contract on the same index (same price level; the tick size can differ: NNQ ticks 0.50 while NQ ticks 0.25).

Appendix B — glossary

Term	Meaning
bar	one row of a candle dataset: open/high/low/close/volume of one interval (1 minute, 1 second)
cache	the engine's in-RAM store of read files and prepared days (section 13)
CVD	cumulative volume delta: running sum of (aggressive buys – aggressive sells)
data root	a folder holding the whole data tree (parquet/ , trades/ , ...)
dataset	one folder of <code>YYYY-MM-DD.parquet</code> day files
DATA slot	a key of DATA : <code>"main"</code> or an additional dataset
front month	the most traded contract of a future on a given day
Globex session	18:00 (previous day) → 17:00 New York, the electronic trading day
look-ahead	using information that wasn't available yet at decision time
points	the instrument's price units (ES: index points)
prepared day	what <code>prepare_day</code> returned for a day, cached
RTH	regular trading hours, 09:30–16:00 New York for equity index futures
RTH date	the calendar date of a session's RTH part, which is the day file's name
tick	the minimum price increment (<code>tick_size</code> , in points)
VWAP	volume-weighted average price